

Deliverable 2 — Architecture Justification

Customizable Subscription Box Scheduler • Problem Statement #35 | Retail, E-Commerce & Finance

Name	SRN	Course
Prajwal G Devadiga	PES1UG24CS590	Software Engineering - Lab 3

Architecture Selection: We chose Microservices Architecture for the Customizable Subscription Box Scheduler System.

Architectural styles considered

Style	Structure	Strength for this scenario	Why it was / was not chosen
Layered	Presentation / business / data tiers in one deployable.	Simple to build and reason about; one codebase to deploy.	The whole tier scales together. The manifest export (10,000 labels in under 60 s) would force the customization portal to scale with it.
Client-Server	Thin clients against one central server.	Centralised control, trivially consistent data.	One server is a single point of failure — an outage during the 48-hour customization window breaks NFR-002's 99.9% availability target.
Microservices (selected)	Independently deployable services per business capability.	Each service scales and fails on its own; payment code is isolated.	More operational overhead and network latency between services.

Two reasons for choosing Microservices

1. The workloads are bursty and on different schedules. The Fulfillment Service must export 10,000 shipping labels in under 60 seconds on dispatch day (NFR-001), while the Subscriber Portal and Order Manager peak in the 48 hours before each renewal (FR-001). As separate services, the Fulfillment Service can be scaled out for the dispatch window alone, instead of over-provisioning the entire application to survive one nightly batch. Each service also owns its own data store, so scaling one never contends for another's database connections.

2. Failure in billing must not close the customization window. NFR-002 requires 99.9% monthly availability for the customization service. If payment handling shared a process with the portal, a slow or failing Payment Gateway on renewal day would consume threads and degrade customization at exactly the moment subscribers are using their 48-hour window. Keeping the Payment Service separate means a gateway outage degrades billing only, and the portal stays up.

Security advantage

Payment credentials and gateway tokens live in exactly one component. The Payment Service is the only component that holds gateway credentials and the only one permitted to open an outbound TLS 1.2+ connection to the external Payment Gateway; it exposes just `IPaymentProcessing` to the Order Manager and accepts no inbound traffic from the public internet. The Subscriber Portal never sees card data — it only ever references a gateway-issued token. This confines cardholder data to a single deployable, so compromising the internet-facing portal does not expose payment credentials, and PCI DSS scope is reduced to one service rather than the whole system.

Performance benefit

The manifest and label export can be scaled horizontally and independently. Because the Fulfillment Service is its own deployable reading box data through `IManifestData` rather than sharing the portal's process, additional instances can be started for the dispatch window to meet NFR-001's 60-second target for 10,000 labels, then scaled back down. The export therefore competes with nothing else for CPU, and subscribers browsing the catalogue during the run see no slowdown — which would not hold in a layered design where both paths share one runtime.